

CIC – 209: DATA STRUCTURES

Unit – I Stacks and Queues

Stack

- A stack is also an ordered collection of elements like arrays, but it has a special feature that deletion and insertion of elements can be done only from one end called the top of the stack (TOP).
- A stack may be seen in our daily life, e.g., a stack of books, a stack of cups, etc.
- It could be thought of just like a stack of plates placed on table in a party, a guest always takes off a fresh plate from the top and the new plates are placed on to the stack at the top.
- It concludes that the item added last will be the item removed first.
- Therefore, stacks are also called **LIFO (Last In First Out)** or FILO (First In Last Out) lists.
- We also call these lists as “piles” or “push-down list”.
- It is a non-primitive data structure.
- When an element is inserted into a stack or removed from the stack, its base remains fixed where the top of stack changes.
- The most accessible element is the top and the least accessible element is the bottom of the stack.

Stack (Contd...)

- “Push” is the term used to insert an element into the stack. “Pop” is the term used to delete an element from the stack.
- The two basic operations associated with stacks are:
 - **Push:** is the term used to insert an element into the stack.
 - **Pop:** is the term used to delete an element from the stack.
- To use a stack efficiently, we need to check the status of stack as well. For the same purpose, the following functionality is added to stacks –
 - **peek()** – get the top data element of the stack, without removing it.
 - **isFull()** – check if stack is full (overflow).
 - **isEmpty()** – check if stack is empty (underflow).
- The stack can be implemented into two ways:
 - Using arrays (Static implementation)
 - Using linked list or pointer (Dynamic implementation)

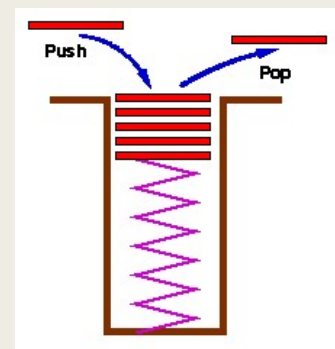
Stack ADT

Objects:

A finite sequence of nodes

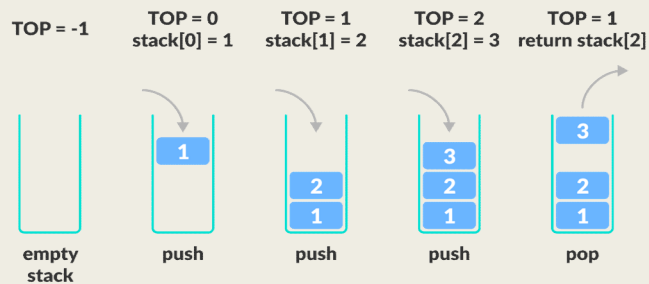
Operations:

- Create
- Push: Insert element at top
- Pop: Remove and return top element
- Peek or Top: Return top element
- isEmpty: test whether stack is empty or not
- isFull: test whether stack is full or not



Representation of Stack

- Let us consider a stack with N elements capacity. This is called as the size of the stack. The number of elements to be added should not exceed the maximum size of the stack. If we attempt to add new element beyond the maximum size, we will encounter a *stack overflow* (*stack full*) condition. Similarly, you cannot remove elements beyond the base of the stack. If such is the case, we will reach a *stack underflow* (*stack empty*) condition.



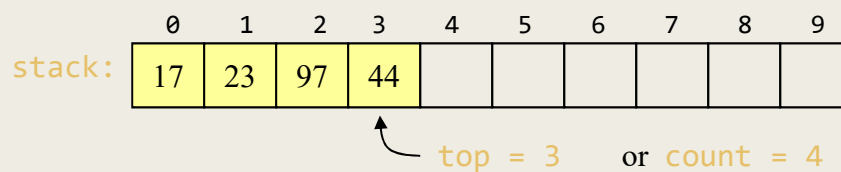
Applications of Stack

- Function call stack
- Removing recursion
- Balancing symbols (parentheses)
- Evaluating Reverse Polish Notation
- Infix to Postfix /Prefix conversion
- Redo-undo features at many places like editors, photoshop.
- Forward and backward feature in web browsers
- Used in many algorithms like Tower of Hanoi, tree traversals, stock span problem, histogram problem.
- Other applications can be Backtracking, Knight tour problem, rat in a maze, N queen problem and sudoku solver
- In Graph Algorithms like Depth First Search, Topological Sorting and Strongly Connected Components

Array implementation of stacks

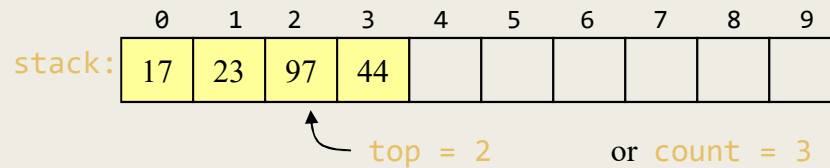
- To implement a stack, items are inserted and removed at the same end (called the top)
- Efficient array implementation requires that the top of the stack be towards the center of the array, not fixed at one end
- To use an array to implement a stack, you need both the array itself and an integer
 - *The integer tells you either:*
 - Which location is currently the top of the stack, or
 - How many elements are in the stack

Pushing and popping



- If the bottom of the stack is at location 0, then an empty stack is represented by top = -1 or count = 0
- To add (push) an element, either:
 - Increment top and store the element in stack[top], or
 - Store the element in stack[count] and increment count
- To remove (pop) an element, either:
 - Get the element from stack[top] and decrement top, or
 - Decrement count and get the element in stack[count]

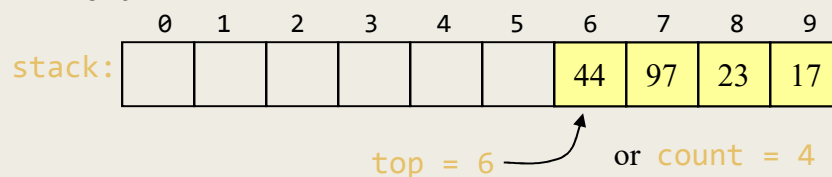
After popping



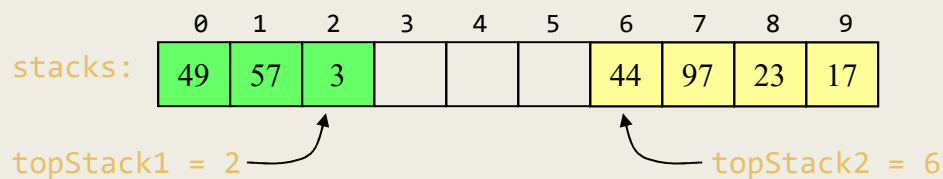
- When you pop an element, do you just leave the “deleted” element sitting in the array?
- The surprising answer is, “it depends”
 - If this is an array of primitives, or if you are programming in C or C++, then doing anything more is just a waste of time
 - If you are programming in Java, and the array contains objects, you should set the “deleted” array element to *null*
 - Why? To allow it to be garbage collected!

Sharing space

- Of course, the bottom of the stack could be at the *other* end



- Sometimes this is done to allow two stacks to share the same storage area



Error checking

- There are two stack errors that can occur:
 - *Underflow: trying to pop (or peek at) an empty stack*
 - *Overflow: trying to push onto an already full stack*

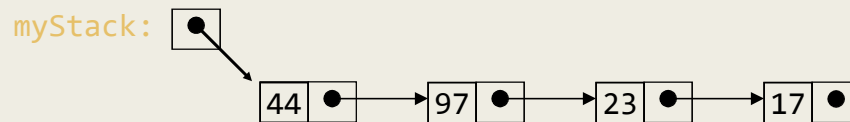
Performance and Limitations

- array-based implementation of stack ADT

- Performance
 - Let n be the number of elements in the stack
 - The space used is $O(n)$
 - Each operation runs in time $O(1)$
- Limitations
 - The maximum size of the stack must be defined a priori, and cannot be changed
 - Trying to push a new element into a full stack causes an implementation-specific exception

Linked-list implementation of stacks

- Since all the action happens at the top of a stack, a singly-linked list (SLL) is a fine way to implement it
- The header of the list points to the top of the stack



- Pushing is inserting an element at the front of the list
- Popping is removing an element from the front of the list

Linked-list implementation details

- With a linked-list representation, overflow will not happen (unless you exhaust memory, which is another kind of problem)
- Underflow can happen, and should be handled the same way as for an array implementation
- When a node is popped from a list, and the node references an object, the reference (the pointer in the node) does *not* need to be set to `null`
 - Unlike an array implementation, it really is removed—you can no longer get to it from the linked list
 - Hence, garbage collection can occur as appropriate

Queue

- A queue is also an ordered collection of elements like arrays, but it has a special feature that the deletion of elements can be done from beginning of the queue (FRONT) and insertion of elements can be done at the end of the queue (REAR).
- A queue may be seen in our daily life, e.g., a queue of persons, a queue of processes, etc.
- The people standing in a railway reservation row are an example of queue. Each new person comes and stands at the end of the row i.e. REAR and person getting their reservation confirmed get out of the row from the front end (FRONT).
- Therefore, queues are also called **FIFO (First In First Out)** or LIFO (Last In Last Out) lists.
- It is a non-primitive data structure.

Queue (Contd...)

- “Enqueue” is the term used to insert an element into the queue. “Dequeue” is the term used to delete an element from the queue.
- The two basic operations associated with queues are:
 - **Enqueue:** is the term used to insert an element into the queue.
 - **Dequeue:** is the term used to delete an element from the queue.
- To use a queue efficiently, we need to check the status of queue as well. For the same purpose, the following functionality is added to queues –
 - **front()** – get the front data element of the queue, without removing it.
 - **isFull()** – check if queue is full (overflow).
 - **isEmpty()** – check if queue is empty (underflow).
- The queue can be implemented into two ways:
 - Using arrays (Static implementation)
 - Using linked list or pointer (Dynamic implementation)

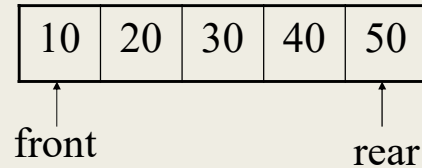
Queue ADT

Objects:

A finite sequence of nodes

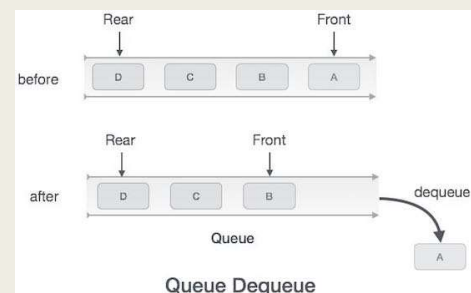
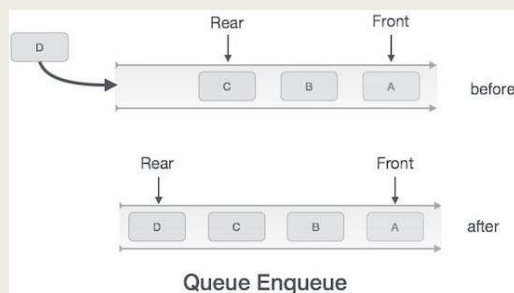
Operations:

- Create
- Enqueue: Insert element at REAR
- Dequeue: Remove and return FRONT element
- Front: Return front element
- isEmpty: test whether queue is empty or not
- isFull: test whether queue is full or not
- size(): returns the number of elements stored



Representation of Queue

- Let us consider a queue with **N** elements capacity. This is called as the size of the queue. The number of elements to be added should not exceed the maximum size of the queue. If we attempt to add new element beyond the maximum size, we will encounter a *queue overflow (queue full)* condition. Similarly, you cannot remove elements beyond the number of elements present in the queue. If such is the case, we will reach a *queue underflow (queue empty)* condition.

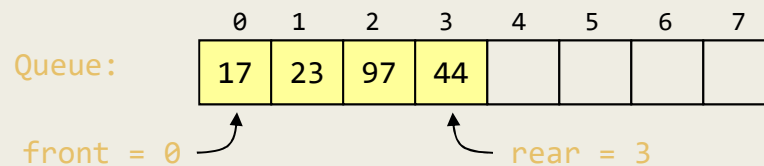


Applications of Queue

- Store packets on network routers
- Hold memory “freelists”
- Make waitlists fair
- Breadth first search
- Serving requests on a single shared resource, like a printer, CPU task scheduling etc.
- In real life scenario, Call Center phone systems uses Queues to hold people calling them in an order, until a service representative is free.
- Handling of interrupts in real-time systems. The interrupts are handled in the same order as they arrive i.e First come first served.

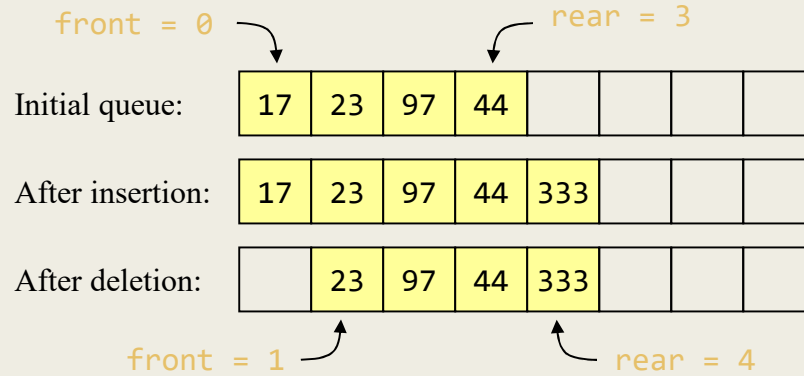
Array implementation of queues

- A queue is a first in, first out (FIFO) data structure
- This is accomplished by inserting at one end (the rear) and deleting from the other (the front)



- **To insert:** put new element in location 4, and set rear to 4
- **To delete:** take element from location 0, and set front to 1

Array implementation of queues



- Notice how the array contents “crawl” to the right as elements are inserted and deleted
- This will be a problem after a while!

Performance and Limitations

- array-based implementation of queue ADT

■ Performance

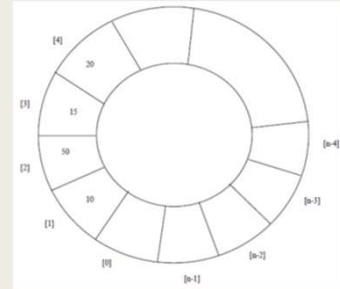
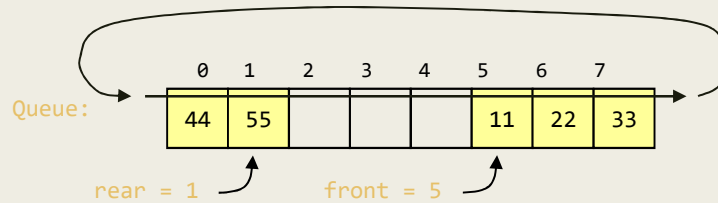
- Let n be the number of elements in the queue
- The space used is $O(n)$
- Each operation runs in time $O(1)$

■ Limitations

- The maximum size of the queue must be defined a priori, and cannot be changed
- Trying to enqueue an element into a full queue causes an implementation-specific exception

Circular Queue

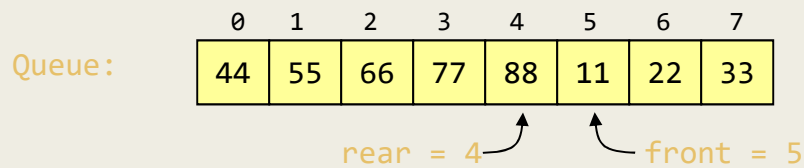
- We can treat the array holding the queue elements as circular (joined at the ends)



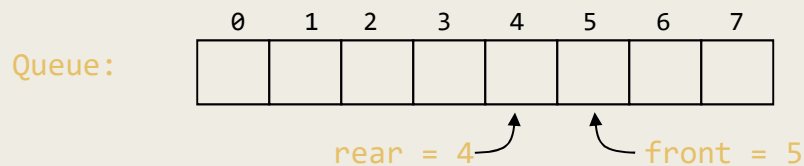
- Elements were added to this queue in the order 11, 22, 33, 44, 55, and will be removed in the same order
- Use: $\text{front} = (\text{front} + 1) \% \text{size};$
and: $\text{rear} = (\text{rear} + 1) \% \text{size};$

Full and empty queues

- If the queue were to become completely full, it would look like this:



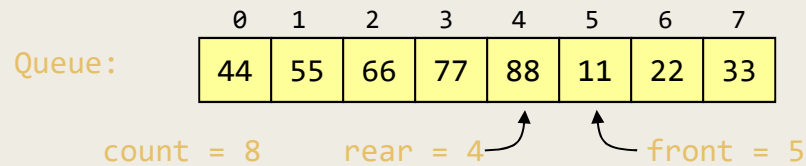
- If we were then to remove all eight elements, making the queue completely empty, it would look like this:



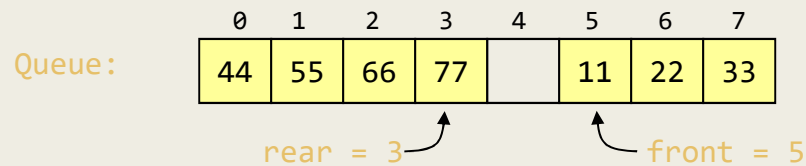
This is a problem!

Full and empty queues: solutions

- **Solution #1:** Keep an additional variable



- **Solution #2:** (Slightly more efficient) Keep a gap between elements: consider the queue full when it has $n-1$ elements



Linked-list implementation of queues

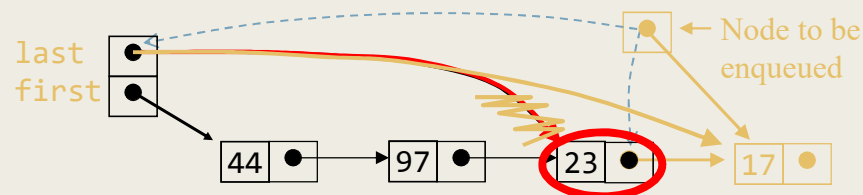
- In a queue, insertions occur at one end, deletions at the other end
- Operations at the front of a singly-linked list (SLL) are $O(1)$, but at the other end they are $O(n)$
 - Because you have to find the last element each time
- BUT: there is a simple way to use a singly-linked list to implement both insertions and deletions in $O(1)$ time
 - You always need a pointer to the first thing in the list
 - You can keep an additional pointer to the last thing in the list

SLL implementation of queues

- In an SLL you can easily find the successor of a node, but not its predecessor
 - Remember, pointers (references) are one-way
- If you know where the *last* node in a list is, it's hard to remove that node, but it's easy to add a node after it
- Hence,
 - Use the first element in an SLL as the front of the queue
 - Use the last element in an SLL as the rear of the queue
 - Keep pointers to both the front and the rear of the SLL

27

Enqueueing a node



To enqueue (add) a node:

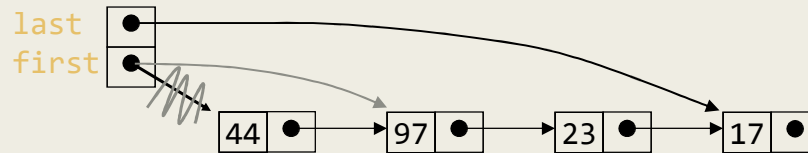
Find the current last node

Change it to point to the new last node

Change the **last** pointer in the list header

28

Dequeuing a node



- To dequeue (remove) a node:
 - Copy the pointer from the first node into the header

29

Queue implementation details

- With an array implementation:
 - you can have both overflow and underflow
 - you should set deleted elements to *null*
- With a linked-list implementation:
 - you can have underflow
 - overflow is a global out-of-memory condition
 - there is no reason to set deleted elements to *null*

30

Double-ended Queues (Dequeues)

- A Double Ended Queue is in short called as Deque (pronounced as Deck or dequeue).
- A deque is a linear queue in which insertion and deletion can take place at either end but not in the middle.
- There are two types of Deque.
 - **Input restricted Deque:** A Deque which allows insertion at only at one end of the list but allows deletion at both the ends of the list is called Input restricted Deque.
 - **Output restricted Deque:** A Deque which allows deletion at only at one end of the list but allows insertion at both the ends of the list is called Output restricted Deque.

31

Implementing Stacks and Queues with Deques

Stacks with Deques:

Stack Method	Deque Implementation
size()	size()
isEmpty()	isEmpty()
top()	last()
push(e)	insertLast(e)
pop()	removeLast()

Queues with Deques:

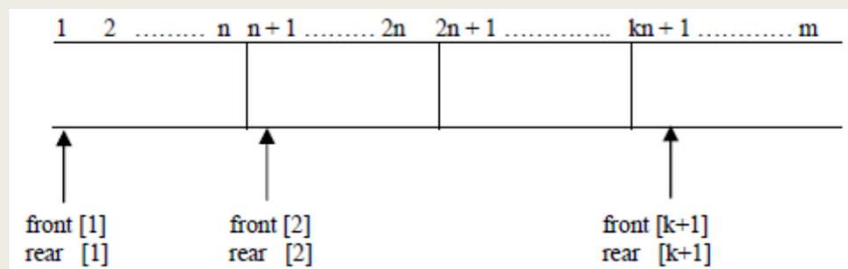
Queue Method	Deque Implementation
size()	size()
isEmpty()	isEmpty()
front()	first()
enqueue()	insertLast(e)
dequeue()	removeFirst()

Priority Queue

- A priority queue is a collection of elements such that each element has been assigned a priority value such that the order in which elements are deleted and processed comes from the following rules:
 - An element of higher priority is processed before any element of lower priority.
 - Two elements with the same priority are processed according to the order in which they were added to the queue.
- A prototype of a priority queue is time sharing system: programs of high priority are processed first, and programs with the same priority form a standard queue. An efficient implementation for the Priority Queue is to use heap, which in turn can be used for sorting purpose called heap sort.
- There are various ways of maintaining a priority queue in memory. One is using a one-way list. The sequential representation is never preferred for a priority queue.

Implementation of Multiple Queues

- Many practical applications in computer science require several queues.
- Multi-queue is a data structure where multiple queues are maintained.
- This type of data structures are used for process scheduling. One may use one dimensional array or multidimensional array to represent a multiple queue.
- A multi-queue implementation using a single dimensional array with m elements is depicted in the Figure. Each queue has n elements which are mapped to a liner array of m elements.



Algebraic Expressions

- An algebraic expression is a legal combination of operators and operands.
- Operand is the quantity on which a mathematical operation is performed.
- Operand may be a variable like x, y, z or a constant like 5, 4, 6 etc.
- Operator is a symbol which signifies a mathematical or logical operation between the operands. Examples of familiar operators include +, -, *, /, ^ etc.
- An algebraic expression can be represented using three different notations:
 - **Infix:** It is the form of an arithmetic expression in which we fix (place) the arithmetic operator in between the two operands. Example: $(A + B) * (C - D)$
 - **Prefix:** It is the form of an arithmetic notation in which we fix (place) the arithmetic operator before (pre) its two operands. The prefix notation is called as polish notation (due to the polish mathematician Jan Lukasiewicz in the year 1920). Example: $* + A B - C D$
 - **Postfix:** It is the form of an arithmetic expression in which we fix (place) the arithmetic operator after (post) its two operands. The postfix notation is called as suffix notation and is also referred to reverse polish notation. E.g.: $A B + C D - *$

Algebraic Expressions (Contd...)

- The three important features of postfix expression are:
 - The operands maintain the same order as in the equivalent infix expression.
 - The parentheses are not needed to designate the expression unambiguously.
 - While evaluating the postfix expression the priority of the operators is no longer relevant.
- Five binary operations: +, -, *, / and ^ or ↑ (exponentiation) are considered. For these binary operations, the following in the order of precedence (highest to lowest):

Operator	Precedence	Value
Exponentiation (^ or ↑)	Highest	3
*, /	Next Highest	2
+, -	Lowest	1

Converting Expressions using Stack

1. Infix to postfix
2. Infix to prefix
3. Postfix to infix
4. Postfix to prefix
5. Prefix to infix
6. Prefix to postfix

Conversion from Infix to Postfix

Procedure to convert from infix expression to postfix expression is as follows:

1. Scan the infix expression from left to right.
2.
 - a) If the scanned symbol is left parenthesis, push it onto the stack.
 - b) If the scanned symbol is an operand, then place directly in the postfix expression (output).
 - c) If the symbol scanned is a right parenthesis, then go on popping all the items from the stack and place them in the postfix expression till we get the matching left parenthesis.
 - d) If the scanned symbol is an operator, then go on removing all the operators from the stack and place them in the postfix expression, if and only if the precedence of the operator which is on the top of the stack is greater than (or greater than or equal) to the precedence of the scanned operator and push the scanned operator onto the stack otherwise, push the scanned operator onto the stack.

Conversion from Infix to Postfix – Example 1

Convert $((A - (B + C)) * D) \uparrow (E + F)$ infix expression to postfix form:

SYMBOL	POSTFIX STRING	STACK	REMARKS
(		(	
(		((	
A	A	((	
-	A	((-	
(	A	(((-	
B	A B	(((-	
+	A B	(((- +	
C	A B C	(((- +	
)	A B C +	(((-	
)	A B C + -	((	
*	A B C + -	((*	
D	A B C + - D	((*	
)	A B C + - D *	((	
↑	A B C + - D *	↑	
(	A B C + - D *	↑ (	
E	A B C + - D * E	↑ (	
+	A B C + - D * E	↑ (+	
F	A B C + - D * E F	↑ (+	
)	A B C + - D * E F +	↑	
End of string	A B C + - D * E F + ↑	The input is now empty. Pop the output symbols from the stack until it is empty.	

Conversion from Infix to Postfix – Example 2

Convert the following infix expression $A + B * C - D / E * H$ into its equivalent postfix expression.

SYMBOL	POSTFIX STRING	STACK	REMARKS
A	A		
+	A	+	
B	A B	+	
*	A B	+ *	
C	A B C	+ *	
-	A B C * +	-	
D	A B C * + D	-	
/	A B C * + D	- /	
E	A B C * + D E	- /	
*	A B C * + D E /	- *	
H	A B C * + D E / H	- *	
End of string	A B C * + D E / H * -	The input is now empty. Pop the output symbols from the stack until it is empty.	

Conversion from Infix to Prefix

- The precedence rules for converting an expression from infix to prefix are identical.
- The only change from postfix conversion is that traverse the expression from right to left and the operator is placed before the operands rather than after them.
- The prefix form of a complex expression is not the mirror image of the postfix form.

Convert the infix expression $A + B - C$ into prefix expression.

SYMBOL	PREFIX STRING	STACK	REMARKS
C	C		
-	C	-	
B	B C	-	
+	B C	- +	
A	A B C	- +	
End of string	- + A B C	The input is now empty. Pop the output symbols from the stack until it is empty.	

Conversion from Infix to Prefix - Example 2

Convert the infix expression $(A + B) * (C - D)$ into prefix expression.

SYMBOL	PREFIX STRING	STACK	REMARKS
)		)	
D	D	)	
-	D	) -	
C	C D	) -	
(	- C D		
*	- C D	*	
)	- C D	*)	
B	B - C D	*)	
+	B - C D	*) +	
A	A B - C D	*) +	
(	+ A B - C D	*	
End of string	* + A B - C D	The input is now empty. Pop the output symbols from the stack until it is empty.	

Conversion from Postfix to Infix

Procedure to convert postfix expression to infix expression is as follows:

1. Scan the postfix expression from left to right.
2. If the scanned symbol is an operand, then push it onto the stack.
3. If the scanned symbol is an operator, pop two symbols from the stack and create it as a string by placing the operator in between the operands and push it onto the stack.
4. Repeat steps 2 and 3 till the end of the expression.

Conversion from Postfix to Infix - Example

Convert the following postfix expression $A B C * D E F ^ / G * - H * +$ into its equivalent infix expression.

Symbol	Stack	Remarks
A	A	Push A
B	A B	Push B
C	A B C	Push C
*	A (B*C)	Pop two operands and place the operator in between the operands and push the string.
D	A (B*C) D	Push D
E	A (B*C) D E	Push E
F	A (B*C) D E F	Push F
^	A (B*C) D (E*F)	Pop two operands and place the operator in between the operands and push the string.
/	A (B*C) (D/(E*F))	Pop two operands and place the operator in between the operands and push the string.
G	A (B*C) (D/(E*F)) G	Push G
*	A (B*C) ((D/(E*F))*G)	Pop two operands and place the operator in between the operands and push the string.
-	A ((B*C) - ((D/(E*F))*G))	Pop two operands and place the operator in between the operands and push the string.
H	A ((B*C) - ((D/(E*F))*G)) H	Push H
*	A (((B*C) - ((D/(E*F))*G)) * H)	Pop two operands and place the operator in between the operands and push the string.
+	(A + (((B*C) - ((D/(E*F))*G)) * H))	
End of string	The input is now empty. The string formed is infix.	

Conversion from Postfix to Prefix

Procedure to convert postfix expression to prefix expression is as follows:

1. Scan the postfix expression from left to right.
2. If the scanned symbol is an operand, then push it onto the stack.
3. If the scanned symbol is an operator, pop two symbols from the stack and create it as a string by placing the operator in front of the operands and push it onto the stack.
4. Repeat steps 2 and 3 till the end of the expression.

Conversion from Postfix to Prefix - Example

Symbol	Stack	Remarks
A	A	Push A
B	A B	Push B
C	A B C	Push C
*	A *BC	Pop two operands and place the operator in front the operands and push the string.
D	A *BC D	Push D
E	A *BC D E	Push E
F	A *BC D E F	Push F
^	A *BC D ^EF	Pop two operands and place the operator in front the operands and push the string.
/	A *BC /D^EF	Pop two operands and place the operator in front the operands and push the string.
G	A *BC /D^EF G	Push G
*	A *BC */D^EFG	Pop two operands and place the operator in front the operands and push the string.
-	A - *BC*/D^EFG	Pop two operands and place the operator in front the operands and push the string.
H	A - *BC*/D^EFG H	Push H
*	A *- *BC*/D^EFGH	Pop two operands and place the operator in front the operands and push the string.
+	+A*- *BC*/D^EFGH	
End of string	The input is now empty. The string formed is prefix.	

Conversion from Prefix to Infix

Procedure to convert prefix expression to infix expression is as follows:

1. Scan the prefix expression from right to left (reverse order).
2. If the scanned symbol is an operand, then push it onto the stack.
3. If the scanned symbol is an operator, pop two symbols from the stack and create it as a string by placing the operator in between the operands and push it onto the stack.
4. Repeat steps 2 and 3 till the end of the expression.

Conversion from Prefix to Infix - Example

Convert the following prefix expression $+ A * - * B C * / D \wedge E F G H$ into its equivalent infix expression.

Symbol	Stack	Remarks
H	H	Push H
G	H G	Push G
F	H G F	Push F
E	H G F E	Push E
^	H G (E^F)	Pop two operands and place the operator in between the operands and push the string.
D	H G (E^F) D	Push D
/	H G (D/(E^F))	Pop two operands and place the operator in between the operands and push the string.
*	H ((D/(E^F))*G)	Pop two operands and place the operator in between the operands and push the string.
C	H ((D/(E^F))*G) C	Push C
B	H ((D/(E^F))*G) C B	Push B
*	H ((D/(E^F))*G) (B*C)	Pop two operands and place the operator in front of the operands and push the string.
-	H ((B*C)-((D/(E^F))*G))	Pop two operands and place the operator in front of the operands and push the string.
*	H (((B*C)-((D/(E^F))*G))*H)	Pop two operands and place the operator in front of the operands and push the string.
A	H (((B*C)-((D/(E^F))*G))*H) A	Push A
+	H (A+(((B*C)-((D/(E^F))*G))*H))	Pop two operands and place the operator in front of the operands and push the string.
End of string		The input is now empty. The string formed is infix.

Conversion from Prefix to Postfix

Procedure to convert prefix expression to postfix expression is as follows:

1. Scan the prefix expression from right to left (reverse order).
2. If the scanned symbol is an operand, then push it onto the stack.
3. If the scanned symbol is an operator, pop two symbols from the stack and create it as a string by placing the operator after the operands and push it onto the stack.
4. Repeat steps 2 and 3 till the end of the expression.

Conversion from Prefix to Postfix - Example

Convert the following prefix expression $+ A * - * B C * / D ^ E F G H$ into its equivalent postfix expression.

Symbol	Stack	Remarks
H	H	Push H
G	H G	Push G
F	H G F	Push F
E	H G F E	Push E
^	H G EF^	Pop two operands and place the operator after the operands and push the string.
D	H G EF^ D	Push D
/	H G DEF^/	Pop two operands and place the operator after the operands and push the string.
*	H DEF^/G*	Pop two operands and place the operator after the operands and push the string.
C	H DEF^/G* C	Push C
B	H DEF^/G* C B	Push B
*	H DEF^/G* BC*	Pop two operands and place the operator after the operands and push the string.
-	H BC*DEF^/G*-	Pop two operands and place the operator after the operands and push the string.
*	BC*DEF^/G*-H*	Pop two operands and place the operator after the operands and push the string.
A	BC*DEF^/G*-H* A	Push A
+	ABC*DEF^/G*-H*+	Pop two operands and place the operator after the operands and push the string.
End of string	The input is now empty. The string formed is postfix.	

Evaluation of Postfix Expression

- The postfix expression is evaluated easily by the use of a stack.
- When a number is seen, it is pushed onto the stack; when an operator is seen, the operator is applied to the two numbers that are popped from the stack and the result is pushed onto the stack.
- When an expression is given in postfix notation, there is no need to know any precedence rules; this is our obvious advantage.

Evaluation of Postfix Expression - Example

Evaluate the following postfix expression: 6 2 3 + - 3 8 2 / + * 2 ↑ 3 +

SYMBOL	OPERAND 1	OPERAND 2	VALUE	STACK
6				6
2				6, 2
3				6, 2, 3
+	2	3	5	6, 5
-	6	5	1	1
3	6	5	1	1, 3
8	6	5	1	1, 3, 8
2	6	5	1	1, 3, 8, 2
/	8	2	4	1, 3, 4
+	3	4	7	1, 7
*	1	7	7	7
2	1	7	7	7, 2
↑	7	2	49	49
3	7	2	49	49, 3
+	49	3	52	52